

HRES Text Analytics

Hybrid Percolator — Elasticsearch ELSER + BM25

Plain English Rules · No Synonyms · No NEAR Queries · 4,000 Rules at Scale

The Simple Explanation

Today, writing a rule to catch burnout complaints looks like this:

```
NEAR((overworked, manager), 5) AND (burnout OR exhausted OR  
"mentally drained") NOT positive
```

With Hybrid Percolator, the same rule looks like this:

"Employee feeling burnt out, exhausted or overworked. Unable to cope with workload or pressure from management."

Plain English. Anyone on the HR team can write it. No query syntax, no synonym files, no distance tuning.

The Core Problem — Why Rules Are Hard Today

Problem	Pain
Rules written in complex query syntax	Only engineers can create or edit them
NEAR/proximity queries	Distance numbers are guesswork — too tight misses things, too loose returns noise
Synonym files	Manually maintained, constantly outdated, never complete
New rule = engineering ticket	Business team blocked, slow iteration

The Solution — ELSER

ELSER (Elastic Learned Sparse Encoder) is Elasticsearch's built-in semantic model. It converts text into a **sparse vector** — a set of weighted terms that capture meaning, not just keywords.

Why ELSER Kills the Synonym Problem

Today you maintain a synonym file like:

burnout, exhausted, drained, worn out, depleted

overworked, overwhelmed, too much work, excessive workload

manager, supervisor, boss, team lead

With ELSER — **you don't need this file at all.**

ELSER learned synonyms during training on billions of documents. It already knows:

- *burnout ≈ exhausted ≈ drained ≈ running on empty*
- *manager ≈ supervisor ≈ boss ≈ team lead*
- *overworked ≈ overwhelmed ≈ too much on my plate*

No file to maintain. No gaps. No updates ever needed.

Why ELSER Replaces NEAR

NEAR was a workaround — a way for keyword search to approximate meaning through word proximity. ELSER understands meaning directly.

"My manager is overworking me" and *"overworked by manager"* are semantically identical to ELSER — word order and distance are irrelevant.

Old Approach	With ELSER
<code>NEAR((overworked, manager), 5)</code>	<i>"employee overworked by manager"</i>
<code>NEAR((burnout, team), 3)</code>	<i>"feeling burnt out within the team"</i>
Synonym file for exhausted/drained/burnout	Not needed
Synonym file for manager/supervisor/boss	Not needed

What is a Percolator?

A regular search query asks: *"which documents match this query?"*

A **Percolator** flips it: *"which queries match this document?"*

Regular Search:

Query → search across documents → matching documents

Percolator:

New document → match against stored queries → matching rules/categories

This is exactly what HRES needs — a feedback comment arrives, and we need to know which of the 4,000 rules it matches.

Hybrid Percolator Architecture

Plain English Rule (written by HR team)

|



ELSER converts to

sparse vector embedding

{burnout: 2.3, exhausted: 1.8,

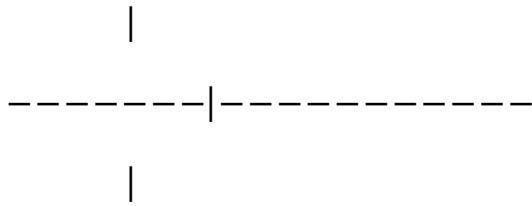
manager: 1.2, workload: 0.9 ...}

|



Stored as percolator query

(sparse_vector match)

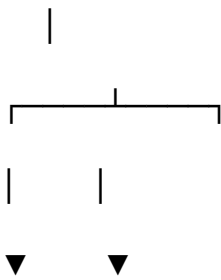


Feedback comment arrives



ELSER embeds feedback

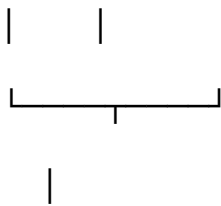
into sparse vector



ELSER BM25

Percolator Percolator

(meaning) (exact terms)



RRF merge





Matched categories stored

in feedback_matches table

The Two Legs — When Each One Wins

ELSER Percolator handles:

- Synonyms and paraphrasing ("*running on empty*" matches *burnout* rule)
- Word order variations ("*manager keeps piling on work*")
- Emotional language ("*I just can't take it anymore*")
- Informal writing ("*team is struggling bad*")

BM25 Percolator handles:

- Exact policy codes (*FMLA, W-2, Form WH-380*)
- Regulatory terms that must match precisely
- Acronyms and system names (*PIP, LOA, EAP*)
- Cases where semantic drift would cause false positives

Feedback	ELSER Catches	BM25 Catches
" <i>I am completely burnt out</i> "	✓	✓ exact word
" <i>Running on empty, can't keep going</i> "	✓ meaning	✗ no keyword
" <i>Manager keeps piling on work</i> "	✓ meaning	✗ no keyword
" <i>Filed FMLA paperwork last week</i> "	⚠ may drift	✓ exact term
" <i>Received a PIP from my supervisor</i> "	⚠ may drift	✓ exact term
" <i>Love it here, great team</i> "	✗ excluded	✗ excluded

The Simplified Rule Builder

With this approach, the rule builder UI becomes simple enough for the HR business team to own directly. No engineers needed.

Screen 1 — Rule List (Home)

HRES Rules Manager

[+ New Rule]

Search rules...

Filter: All Categories ▼

Rule Name	Category	Status	Actions
BURNOUT_COMPLAINT	BURNOUT	✓ Live	Edit Test ...
FMLA_REQUEST	LEAVE	✓ Live	Edit Test ...
PAY_DISPUTE	COMPENSATION	✓ Live	Edit Test ...
HOSTILE_WORKPLACE	GRIEVANCE	⏸ Draft	Edit Test ...
MANAGER_CONFLICT	GRIEVANCE	✓ Live	Edit Test ...
...	...	...	

Showing 1–20 of 4,000 rules

[Next Page]

Screen 2 — Create / Edit Rule

← Back to Rules

Create New Rule

Rule Name

BURNOUT_COMPLAINT

Category

Sub-Category

BURNOUT

▼

WORKLOAD

▼

|
|

|

| ① Describe the rule in plain English ← ELSER reads |

|

— |

|

|

| | Employee feeling burnt out, exhausted or overworked. | |

| | Unable to cope with workload or pressure from | |

| | management. Mentally drained, running on empty. | |

| | | |

|

|

| 💡 Write naturally — synonyms and paraphrasing handled |

| automatically. No query syntax needed. |

|

|

|

|

| ② Must contain (exact terms only) ← BM25 reads |

|

— |

[illegible]

④ Minimum match confidence

Low

70%

High

💡

Higher = fewer but more precise matches

[Cancel]

[Save as Draft]

[Test Rule →]

← Back to Edit

Test Rule: BURNOUT_COMPLAINT

Paste sample feedback to test against this rule:

I've been feeling completely drained lately. My manager keeps adding more to my plate and I just can't keep up.

I dread coming into work every morning.

[Run Test]

Results

MATCH — BURNOUT_COMPLAINT

ELSER score:

0.87

Strong

BM25 score:

0.00

(no exact terms required for this rule)

Combined score:

0.84

Above threshold

Why it matched:

"completely drained" → matched burnout/exhaustion intent

| • "manager keeps adding" → matched workload/pressure pattern |

| • "can't keep up" → matched overwhelmed/overwork |

| • "dread coming to work" → matched disengagement signal |

| |

|
|

| Try another sample: |

|

|
|

| | I love my team and my manager is very supportive. | |

|
|

|

| [Run Test] |

|
|

| Results |

|

|

| |

| ✖ NO MATCH |

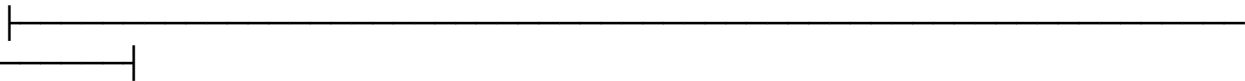
| |

| ELSER score: 0.12  Weak |

| Excluded term: "love" triggered exclusion filter |

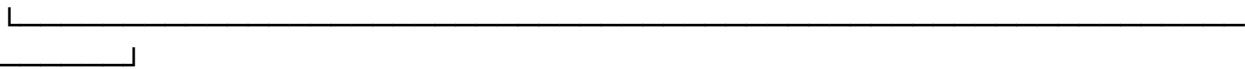
| Combined score: 0.00 Below threshold — correctly excluded |

| |



| |

| [← Adjust Rule] [☒ Publish Rule] |



Screen 4 — Rule Published Confirmation



| |

| ☒ Rule Published |

| |

| BURNOUT_COMPLAINT is now live |

| |

| It will be applied to all new incoming |

| feedback automatically. |

| |

| To apply to existing 1.5M comments: |

| [Run Backfill Job] (est. ~3 hours) |

| |

[Back to Rules List]

What Each Section Does Under the Hood

UI Section	Who Handles It	What Happens
① Plain English description	ELSER	Converted to sparse vector, stored as percolator query
② Exact terms	BM25	Stored as <code>match</code> query in percolator
③ Exclude terms	Both	Applied as NOT filter on both legs
④ Confidence threshold	RRF score	Minimum combined score to trigger a match
Test Rule	Live percolator	Runs the actual query against sample text in real time
Publish	Elasticsearch	Percolator document goes live immediately

No NEAR. No synonym files. No query syntax. Just plain English.

Rule Storage

Each rule stored in Elasticsearch as a percolator document:

Field	Content
<code>rule_id</code>	Unique identifier

Field	Content
rule_name	BURNOUT_COMPLAINT
category	BURNOUT
plain_text_description	Human-readable intent (what HR wrote)
elser_query	Sparse vector percolator query (auto-generated from plain text)
bm25_query	Exact term match for policy codes (if any)
exclude_terms	NOT conditions
rule_version	For staleness tracking

Pre-Computed Matches

Once a feedback record is categorized, matches are stored permanently — the percolator runs **once per feedback at write time**, not on every query.

Feedback arrives (write time)

|



Run Hybrid Percolator once

|



Store matches in feedback_matches index

|



All future reads = simple lookup ← <1ms, zero computation

What Gets Stored per Match

Field	Value
feedback_id	Source feedback identifier
rule_id	Matched rule
category	BURNOUT, GRIEVANCE, COMPENSATION etc.
elser_score	Semantic match score
bm25_score	Keyword match score
rrf_score	Combined final score
matched_at	Timestamp
rule_version	Version at time of match

Handling Rule Changes

When a rule is updated, only the records where that rule previously fired (or was a close candidate) need reprocessing — not all 1.5M records.

Rule updated

|



Increment rule_version

|



Background job finds affected records

(where old rule_version was matched)

|



Reprocess only those records

|



Update matches with new rule_version

Performance — 4,000 Rules at Scale

Approach	Rules Evaluated	Time per Record
Naive — evaluate all rules	4,000	Seconds
Elasticsearch Percolator alone	4,000 (indexed)	~10–30ms
Hybrid Percolator (ELSER + BM25 + RRF)	All 4,000 — natively	~15–40ms
Pre-computed (after first run)	0 — just a lookup	<1ms

Elasticsearch Percolator is purpose-built for matching documents against stored queries at scale — it indexes the queries, not just the documents. 4,000 rules is comfortably within its design envelope.

What You Gain vs Today

What You Have Today	What You Get
Complex query syntax per rule	Plain English descriptions
NEAR distances manually tuned	Not needed — ELSER handles proximity semantically

What You Have Today	What You Get
Synonym file manually maintained	Eliminated — ELSER knows synonyms natively
Only engineers write rules	HR business team writes and edits rules directly
Synonyms go stale over time	ELSER never goes stale
New rule = engineering ticket	New rule = HR types a sentence

Oracle 23ai vs Elasticsearch — Full Comparison

Feature	Oracle 23ai	Elasticsearch
Embedding model	MiniLM (loaded by DBA)	ELSER (built-in, zero setup)
Keyword search	Oracle Text CONTAINS	BM25 match query
Semantic search	VECTOR_DISTANCE	sparse_vector query
Hybrid merge	Manual RRF in SQL CTE	Native rrf — one line
Rule matching at scale	Two-stage pipeline (manual)	Percolator (purpose-built)
Synonym handling	Manual synonym file	Eliminated by ELSER
NEAR / proximity queries	Required	Eliminated by ELSER
Plain English rules	No — requires query syntax	Yes
Setup effort	DBA loads model manually	Works out of the box
Where data lives	Inside Oracle DB	Separate ES cluster
Already approved at BoA	 Yes	Needs approval

Bottom Line

ELSER + Percolator doesn't just improve search — it **changes who owns the rules**. When a rule is a plain English sentence, the HR business team can create, edit, and test rules directly. Engineering builds the pipeline once. The business runs it forever.

The synonym problem disappears. The NEAR problem disappears. What's left is a clean, maintainable, scalable text analytics system that gets smarter as your rules get better — written by the people who understand the domain, not the people who understand query syntax.

HRES Text Analytics — Internal Use Only